Password & Authentication Lab – Practical Questions

Tool 1: John the Ripper (Password Cracking)

Question 1:
You are given a file "passwords.txt" containing hashed passwords.
Task:

- **Identify the hash type**
- **Crack the passwords using John the Ripper**

Example:
Hash: 5f4dcc3b5aa765d61d8327deb882cf99

Steps:

- **Save hash in file: hash.txt**
- **Run: john hash.txt**
- **Show cracked password**

Question 2:
Use a custom wordlist to crack passwords.
Task:

- **Create your own wordlist (at least 20 passwords)**
- **Run John using:**
  john --wordlist=wordlist.txt hash.txt

Question 3:
Analyze Results

- **Which passwords were cracked?**
- **Why were some not cracked?**

Tool 2: Hashcat (Advanced Cracking)

Question 1:
Crack an MD5 hash using Hashcat.

Example:
Hash: 098f6bcd4621d373cade4e832627b4f6

Steps:

- **Save hash in file: hash.txt**
- **Run:**
  hashcat -m 0 -a 0 hash.txt wordlist.txt

Question 2:
Perform a brute-force attack using mask.

Example:
Password format: 4 digits (e.g., 1234)

Command:
hashcat -m 0 -a 3 hash.txt ?d?d?d?d

Question 3:
Compare Performance

- **Time taken by Hashcat vs John**
- **Which is faster and why?**

Tool 3: Hydra (Login Brute Force)

Question 1:
Perform brute-force attack on login form (lab environment only)

Example:
Target: testphp.vulnweb.com

Command:
hydra -l admin -P passwords.txt testphp.vulnweb.com http-post-form "/login.php:username=^USER^&password=^PASS^:F=incorrect"

Task:

- **Identify correct username/password**
- **Capture screenshot**

Question 2:
Create your own login page (local)

- **Use simple HTML form**
- **Run Hydra attack on localhost**

Question 3:
Defense Analysis

- **How can brute-force attacks be prevented?**
  (e.g., rate limiting, CAPTCHA, 2FA)

Final Questions

1. **Why is password hashing important?**
2. **Difference between MD5, SHA1, and bcrypt?**
3. **What makes a password "strong"?**
4. **Why do brute-force attacks fail sometimes?**
5. **How can companies detect password attacks?**

Submission:

- **Screenshots of commands**

Aim

To understand password authentication mechanisms and perform password-cracking and brute-force attack simulations using John the Ripper, Hashcat, and Hydra in a controlled Kali Linux environment.

Objectives

1. Identify password hash types.
2. Crack password hashes using John the Ripper.
3. Create and use a custom wordlist.
4. Perform password-cracking using Hashcat.
5. Compare the performance of John the Ripper and Hashcat.
6. Simulate brute-force attacks using Hydra.
7. Study password security mechanisms and defense techniques.

Tools Required

- Kali Linux
- John the Ripper
- Hashcat
- Hydra
- Terminal
- Firefox Browser

Initial Setup

A working directory named **CyberLab** was created to store all files required for the practical. The following files were created:

- notes.txt
- passwords.txt
- hash.txt
- wordlist.txt

Figure 1: Creation of CyberLab Directory

Figure 2: Creation of Required Files

Figure 3: Verification of Files Using ls Command

Tool 1: John the Ripper
Question 1: Password Hash Cracking

Hash Used
5f4dcc3b5aa765d61d8327deb882cf99

The hash was stored in **hash.txt** and analyzed using John the Ripper. The hash type was identified as **MD5 (Raw-MD5)**.

Command Used
john --format=Raw-MD5 hash.txt

Displaying Cracked Password
john --show --format=Raw-MD5 hash.txt

Observation

John the Ripper successfully identified the hash type and recovered the original password from the MD5 hash.

Result

Recovered Password:

password

Figure 4: MD5 Hash Stored in hash.txt

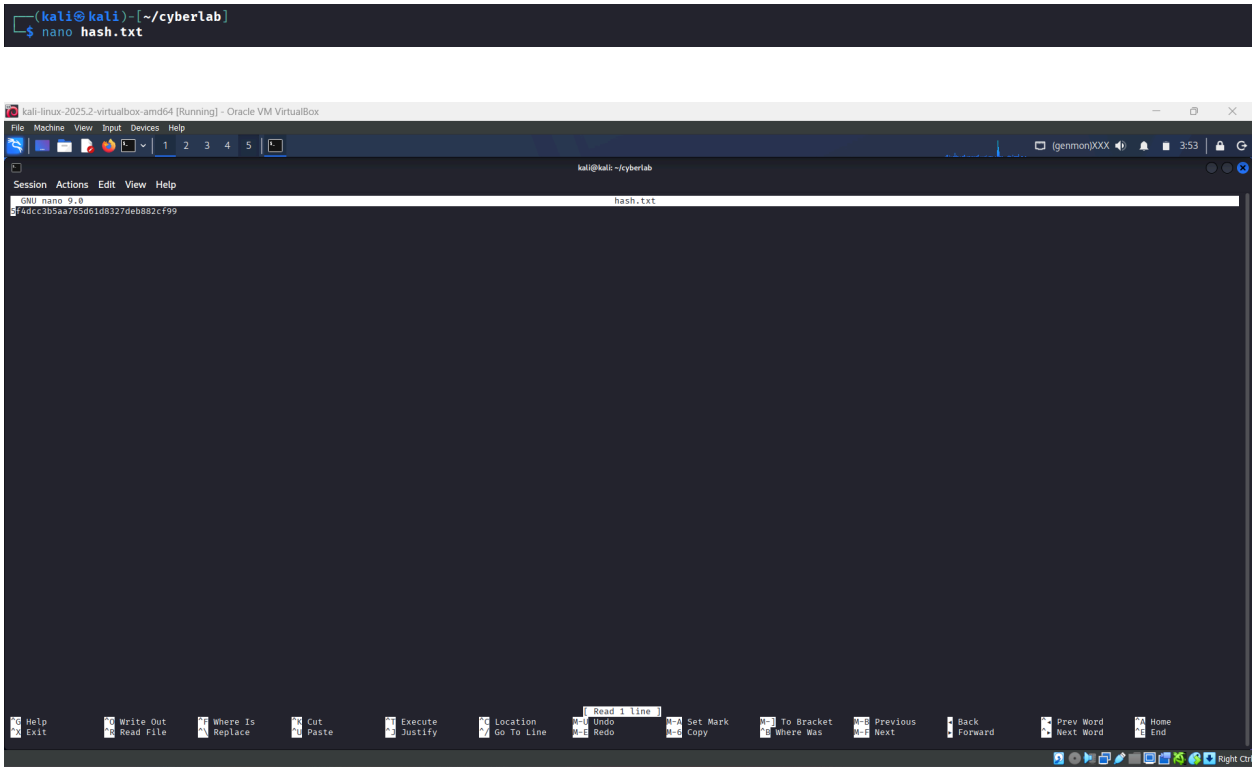


Figure 5: John the Ripper Password Cracking Process

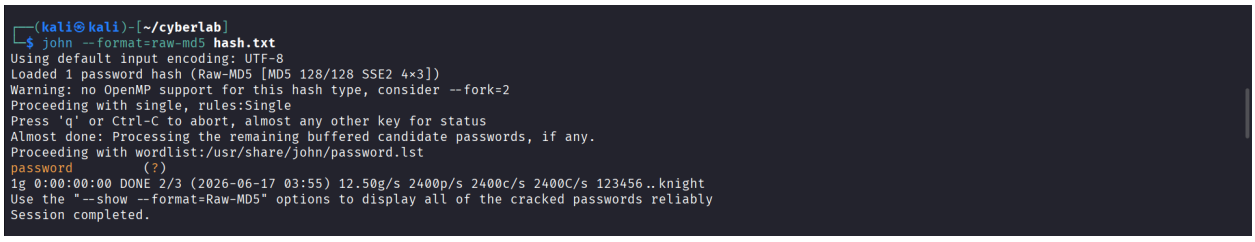


Figure 6: Cracked Password Displayed Using john --show

```
┌──(kali㉿kali)-[~/cyberlab]
└─$ john --show --format=Raw-MD5 hash.txt
?:password

1 password hash cracked, 0 left
```

Question 2: Custom Wordlist Attack

A custom wordlist containing 20 passwords was created and saved in **wordlist.txt**.

Command Used
john --format=Raw-MD5 --wordlist=wordlist.txt hash.txt

Observation

The password was successfully cracked because it was present in the custom wordlist.

Result

Dictionary-based password recovery was successful.

Figure 7: Custom Wordlist Contents

```
┌──(kali㉿kali)-[~/cyberlab]
└─$ nano wordlist.txt
```

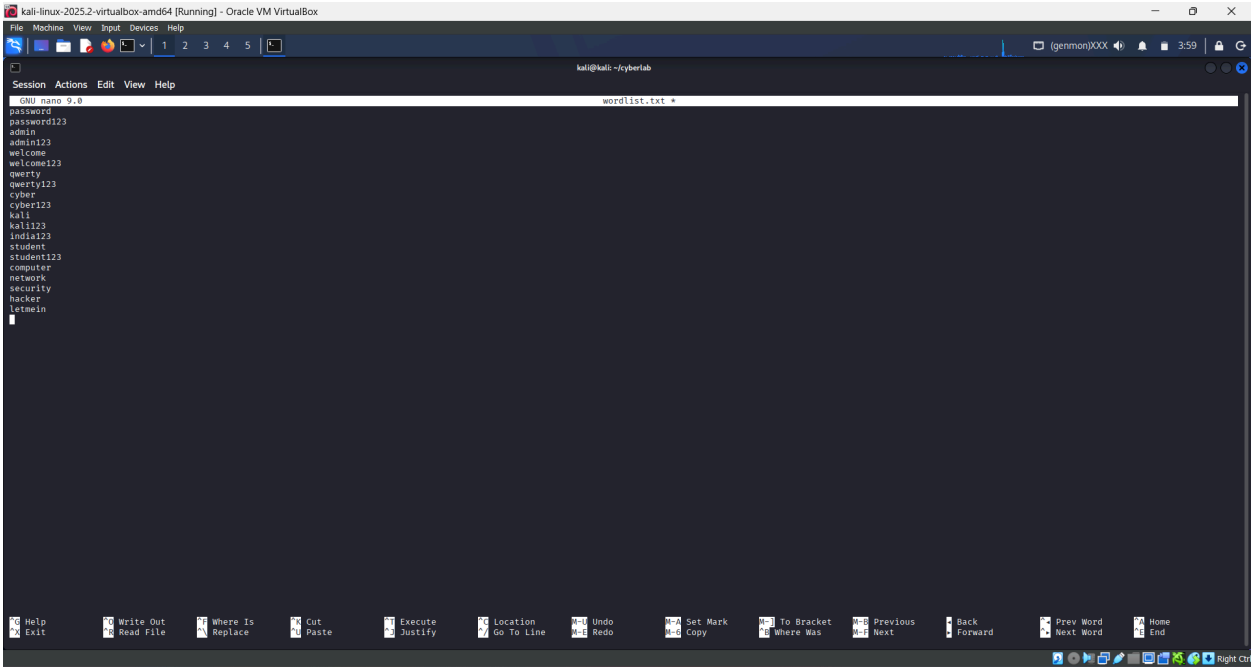


Figure 8: John Using Custom Wordlist

Question 3: Analysis

Which Passwords Were Cracked?

Passwords that existed in the custom wordlist were successfully cracked.

Why Were Some Passwords Not Cracked?

Some passwords may not be cracked because:

- They are not present in the wordlist.
- They contain special characters.
- They are long and complex.
- Additional computational resources may be required.

Tool 2: Hashcat
Question 1: MD5 Hash Cracking

Hash Used
098f6bcd4621d373cade4e832627b4f6

Command Used
hashcat -m 0 -a 0 hash.txt wordlist.txt

Observation

Hashcat was executed to crack the MD5 hash using the custom wordlist. However, the virtual machine reported insufficient OpenCL memory during execution.

Result

The command executed successfully, but password recovery could not be completed due to memory allocation limitations in the execution environment.

Expected Password:

test

Figure 9: Hashcat Dictionary Attack Command

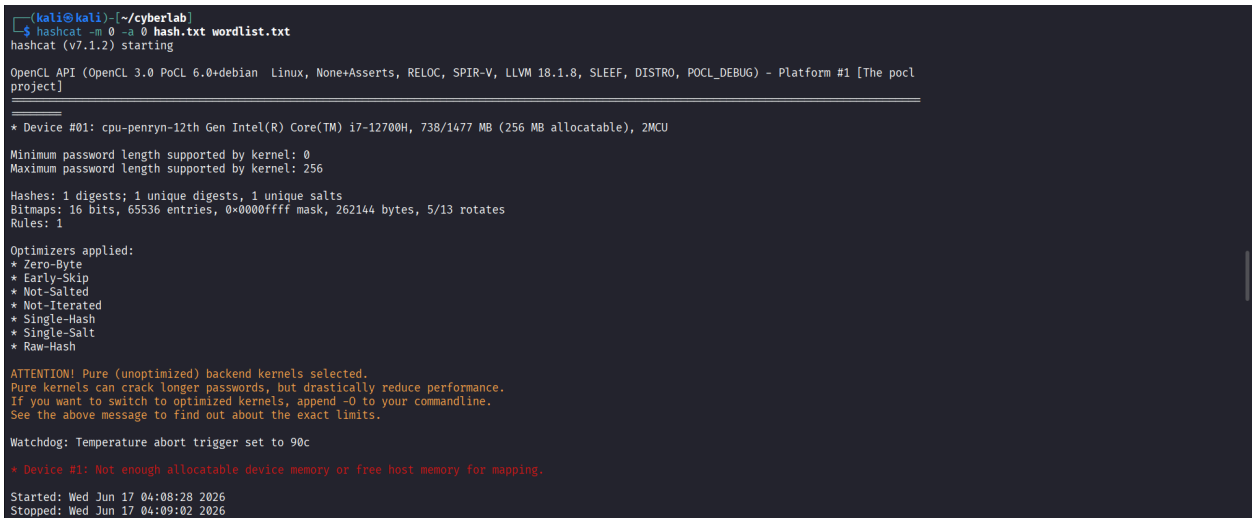


Figure 10: Hashcat Memory Allocation Error

Additional Observation

The following command was executed to display recovered passwords:

hashcat -m 0 hash.txt --show

Since Hashcat was unable to crack the hash due to memory limitations, no recovered password was available for display and the command produced no output.

Question 2: Brute-Force Attack Using Mask

Command Used

hashcat -m 0 -a 3 hash.txt ?d?d?d?d

Observation

A mask attack was performed to test all four-digit numeric combinations from 0000 to 9999. The command executed correctly; however, the same memory allocation issue prevented successful completion of the attack.

Result

The brute-force attack was initiated successfully, but no password recovery result was obtained because of resource limitations.

Figure 11: Hashcat Brute-Force Mask Attack

Question 3: Compare Performance

Feature	John the Ripper	Hashcat
Processing Method	CPU-Based	GPU-Based
Speed	Fast	Very Fast
Ease of Use	Easy	Moderate
Performance	Good	Excellent
Best Use	Password Auditing	High-Speed Cracking

Which is Faster and Why?

Hashcat is generally faster because it supports GPU acceleration. Graphics Processing Units contain thousands of cores capable of processing multiple password combinations simultaneously, allowing Hashcat to perform password-cracking operations significantly faster than CPU-based tools.

Tool 3: Hydra
Question 1: Login Form Brute Force

Target

testphp.vulnweb.com

Command Used

hydra -l admin -P passwords.txt testphp.vulnweb.com http-post-form "/login.php:username=^USER^&password=^PASS^:F=incorrect"

Observation

Hydra initiated the brute-force attack successfully. During execution, multiple connection errors occurred, preventing the tool from testing all credentials against the target website.

Result

No valid username/password combination was identified due to connection failures during testing.

Figure 12: Hydra Web Login Attack Command

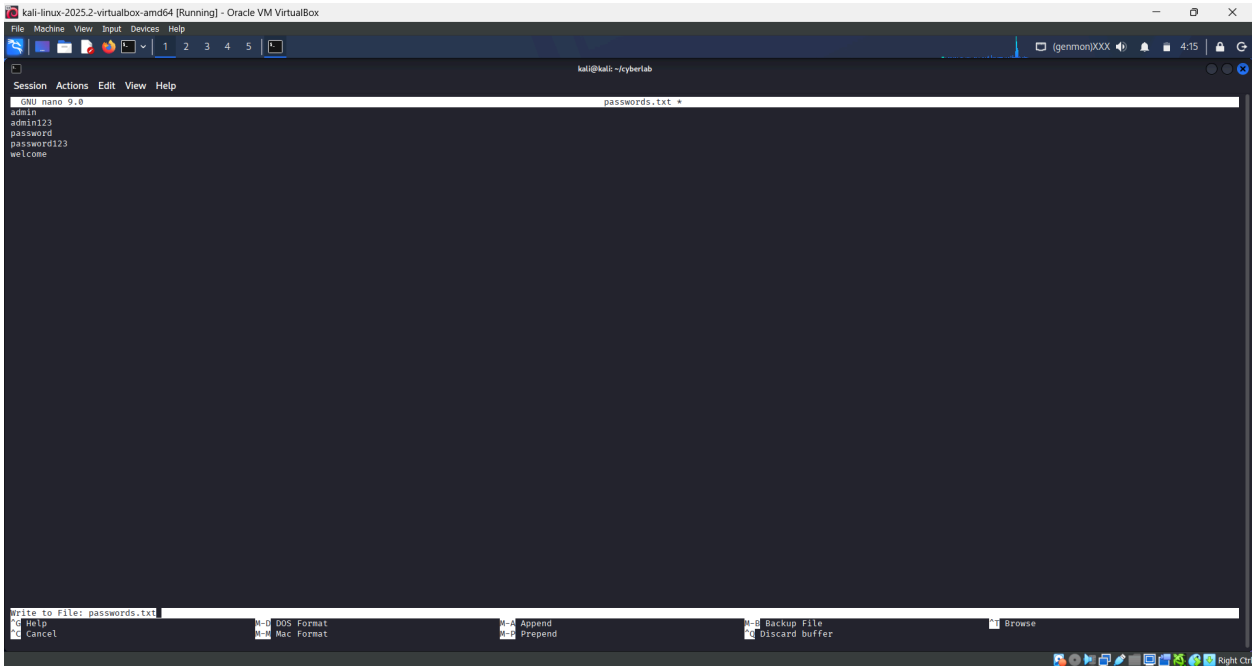
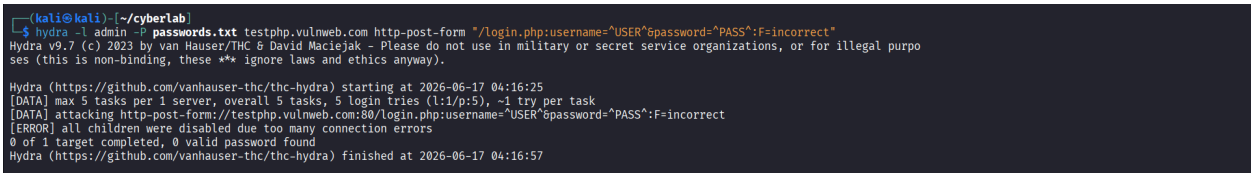


Figure 13: Hydra Connection Error Output



Question 2: Local Login Page

A simple HTML login page was created for authentication testing.

HTML Code

```
<html>
<body>
<form>
Username: <input type="text">
Password: <input type="password">
<input type="submit" value="Login">
</form>
</body>
</html>
```

Observation

The local login page was created successfully and opened in the browser for testing purposes.

Figure 14: HTML Login Page Source Code

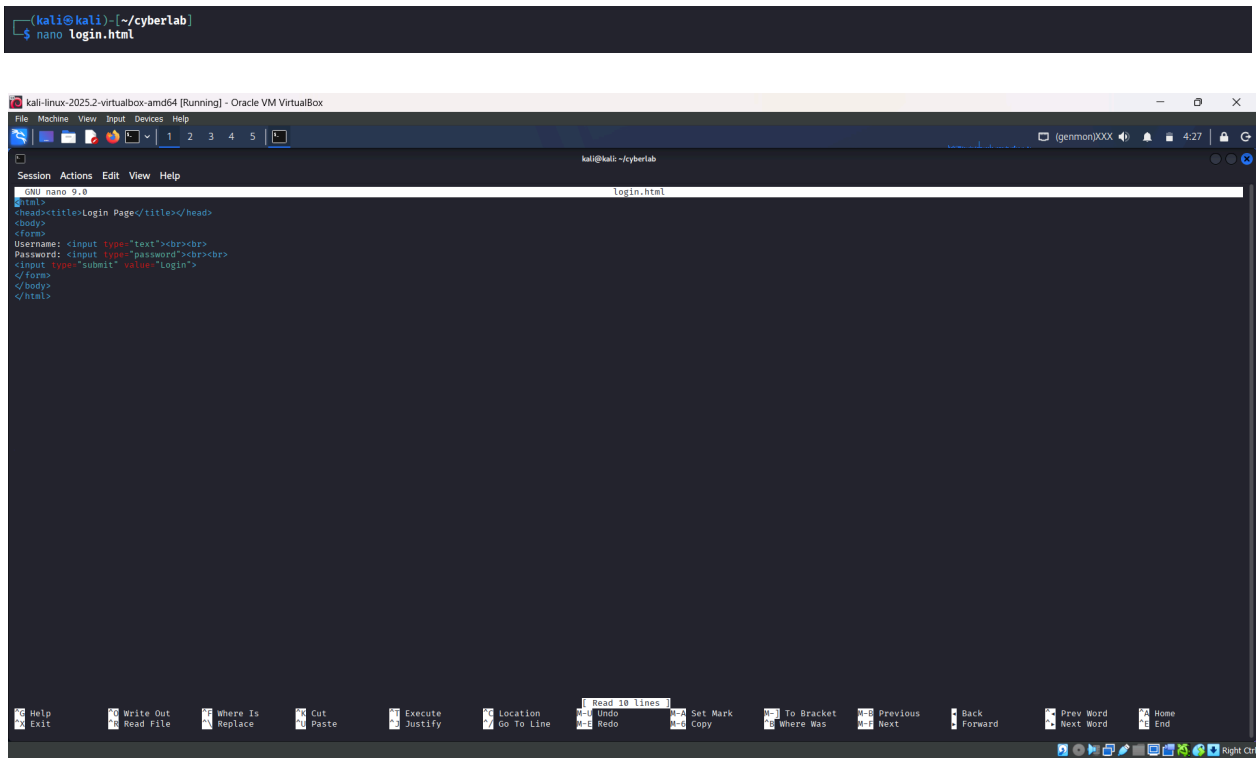
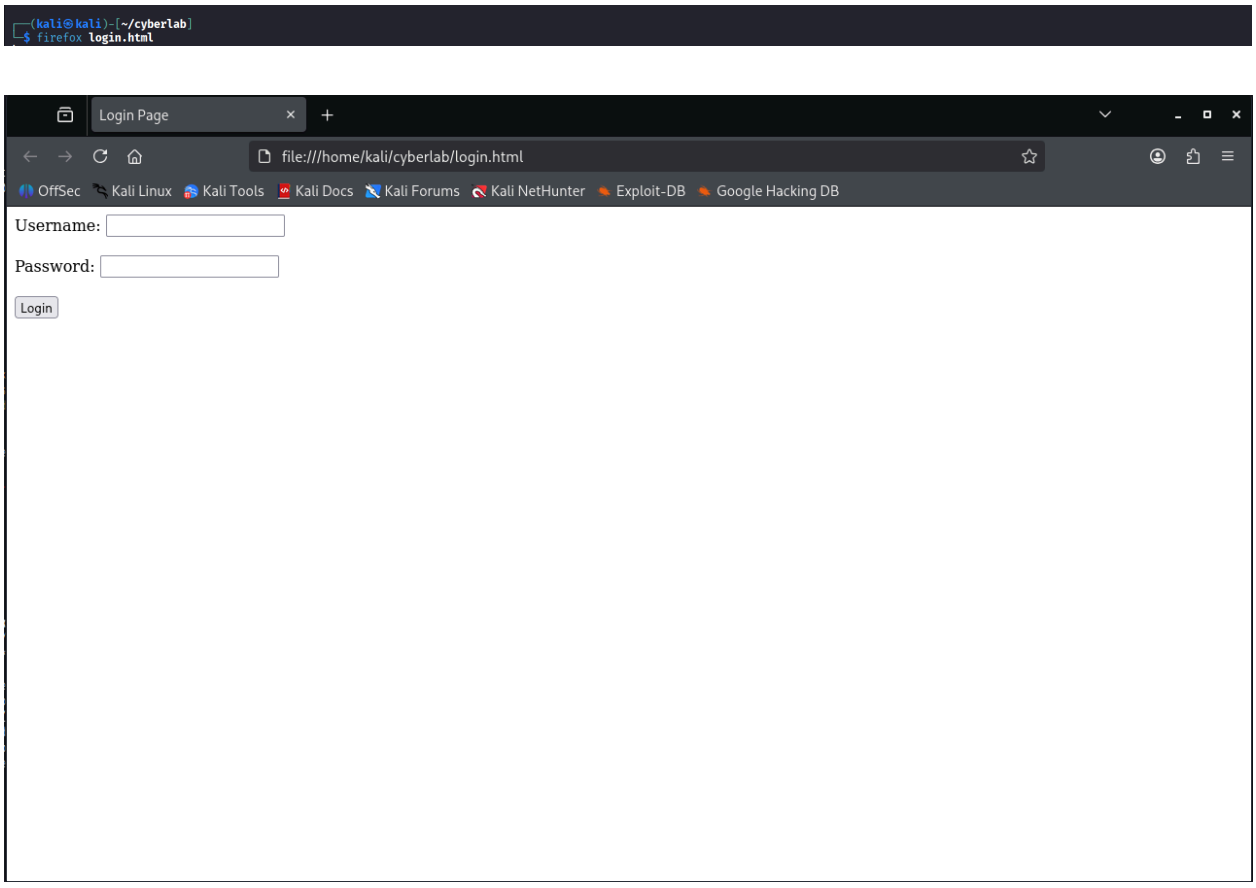


Figure 15: Login Page Displayed in Browser



Hydra Attack on Localhost SSH

Command Used
hydra -l kali -P passwords.txt ssh://127.0.0.1

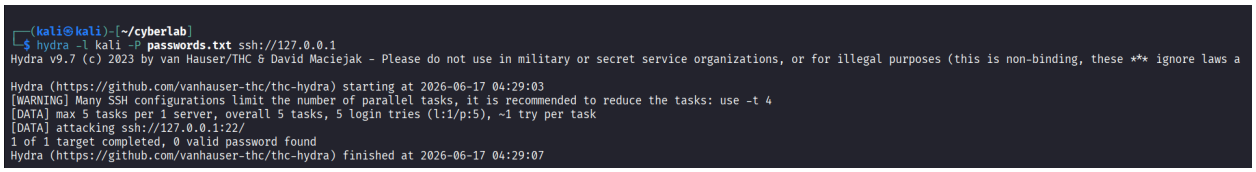
Observation

Hydra successfully connected to the local SSH service and tested passwords from the custom password list.

Result

The attack completed successfully; however, no valid password was found because the correct password was not present in the supplied wordlist.

Figure 17: Hydra SSH Attack Result



Defense Analysis

Brute-force attacks can be prevented using multiple security controls:

1. Rate Limiting
2. CAPTCHA
3. Account Lockout Mechanisms
4. Multi-Factor Authentication (MFA)
5. Strong Password Policies
6. Login Monitoring and Logging
7. Web Application Firewalls (WAF)

These mechanisms reduce the effectiveness of automated password attacks and improve overall system security.

Final Questions

Why is Password Hashing Important?

Password hashing converts passwords into fixed-length values before storage. Instead of storing the actual password, systems store its hash value, making it difficult for attackers to recover original credentials even if a database is compromised.

Difference Between MD5, SHA1, and bcrypt

MD5	SHA1	bcrypt
128-bit hash	160-bit hash	Adaptive password hashing algorithm
Fast	Fast	Intentionally slow
Vulnerable to attacks	Vulnerable to attacks	Highly secure
Not recommended	Not recommended	Recommended for password storage

What Makes a Password Strong?

A strong password:

- Contains at least 12 characters.
- Includes uppercase and lowercase letters.
- Contains numbers and symbols.
- Avoids personal information.
- Is unique for every account.

Why Do Brute-Force Attacks Fail Sometimes?

Brute-force attacks may fail because:

- Passwords are highly complex.
- CAPTCHA blocks automated attempts.
- Account lockout policies are enabled.
- Multi-factor authentication is used.
- The attacker lacks sufficient resources.

How Can Companies Detect Password Attacks?

Organizations detect password attacks through:

- Failed login monitoring
- Intrusion Detection Systems (IDS)
- Security logs
- SIEM platforms
- Behavioral analysis
- Alerting mechanisms

Challenges Faced and Solutions

During the practical, several challenges were encountered. While using Hashcat, the virtual machine reported insufficient OpenCL memory, preventing successful completion of both dictionary and brute-force attacks. Although the commands executed correctly, the expected password recovery output could not be obtained due to hardware and resource limitations of the execution environment.

During the Hydra web login attack against the test website, repeated connection errors occurred. These errors likely resulted from server-side protections such as rate limiting, firewall restrictions, or changes in the target application's configuration. Consequently, no valid credentials could be identified from the online target.

To continue the practical despite these limitations, a local SSH-based Hydra simulation was performed. The attack completed successfully, demonstrating Hydra's functionality; however, no valid password was found because the correct credential was not present in the supplied password list.

These challenges provided valuable insight into both the practical limitations of password-cracking tools and the effectiveness of modern security mechanisms used to defend against unauthorized access attempts.

Conclusion

This practical provided hands-on experience with password authentication and password-cracking tools including John the Ripper, Hashcat, and Hydra. Password hashes were analyzed using dictionary-based and brute-force techniques, while login authentication mechanisms were examined through controlled attack simulations. The practical highlighted the importance of strong passwords, secure hashing algorithms, account protection mechanisms, and multi-factor authentication in defending systems against unauthorized access and password-based attacks.